Resource Isolation in MoQT Relays

Progress Report II - Group 16
Kaustubh Bhal, Shradha Gandotra, Rudra Kanani, Sanat Dhanyamraju, Alan Zhu
November 10, 2026

1. Problem Definition

Our project studies how a Media over QUIC Transport (MoQT) relay should share memory and outgoing bandwidth among subscribers. A subscriber that stops keeping up can leave old media waiting for delivery. We want to stop that backlog from affecting other clients while helping the stalled subscriber return to current media.

We have built a local testbed and implemented two policies: a fixed timeout that ends stalled subscriptions, and a policy that skips stale media streams while keeping the subscription open. We also added a shared bandwidth limit so the comparison includes real contention for a controlled resource. Compared with Report I, our completed work focuses on stalled-client recovery under a shared bandwidth limit. The DRR scheduler, per-session byte budgets, and global application-memory cap are not implemented yet; multi-track subscribers remain untested.

2. Motivation

For live media, receiving every byte eventually is not enough. Our uncapped baseline showed paused subscribers receiving about 10 Mbit/s after resuming, even though the source produced only 4 Mbit/s. Much of that traffic was old backlog. When outgoing bandwidth is limited, this catch-up traffic can delay media for healthy clients too.

Closing slow subscriptions protects other clients, but gives up on those subscribers. We therefore tested whether dropping stale media could preserve that protection while letting the same subscribers recover.

3. Challenges

The first challenge was creating real backpressure. Slowing only the application's reader would let the client library keep buffering data. Our harness instead pauses MoQT session processing while QUIC continues transport processing and acknowledgments. All clients use finite receive windows and one incoming media stream at a time. The single-stream limit avoids an overlapping-group recovery failure observed in the pinned library, but also limits how broadly we can generalize the results.

Skipping a stream introduces another challenge: an intentional reset can interrupt an Object halfway through. We changed the receiver to preserve actual reset codes, so the client can record that loss and continue without accepting corrupt or truncated Objects as successful delivery.

Finally, all processes share one machine. We excluded earlier runs with timing gaps consistent with host suspension and added a duration check. Local tests provide controlled comparisons, but do not reproduce separate client links, video decoding, or a deployed relay.

4. Methodology and Implementation Progress

We pinned Cloudflare's relay to draft-16 revision `3b2b584`. The original baseline uses its unchanged source. Our modifications are separate, reproducible patches, and each comparison uses matched relay and client binaries with recorded hashes.

The Rust load generator publishes one synthetic track through direct `PUBLISH`. Each Object contains a sequence number, generation timestamp, and checkable payload. This gives us known offered load and complete-Object latency measurements. We chose synthetic media Objects because they let us repeat the same workload and measure how much delivered data is still timely. Unlike a speed test or request-response benchmark, this workload continuously sends one source to many subscribers. It exercises shared downstream fan-out; it does not test upstream `SUBSCRIBE` deduplication or encoded-video playback. The variable-size keyframe workload and Big Buck Bunny replay proposed in Report I remain future work.

Setting	**Local platform and workload**
Hardware	Apple M2 Pro; 10 CPUs; 16 GiB RAM
Operating system	macOS 15.3.1, ARM64
Processes	One relay, one publisher, one subscriber generator; four runtime worker threads each
Source	30 Objects/s, 16,666 bytes each; approximately 4 Mbit/s; one new group per second
Subscribers	Independent QUIC sessions; 16 in policy comparisons, 16 or 64 in the initial baseline
Receive limits	64 KiB per connection, 32 KiB per stream, one incoming unidirectional stream
Bandwidth limit	80 Mbit/s of shared outgoing UDP payload, with a 64 KiB burst allowance
Replication	Five runs per condition, with condition order shuffled within each repetition

Sixteen full-rate subscribers need about 64 Mbit/s of media before overhead. The 80 Mbit/s limit leaves room for normal traffic but makes catch-up traffic compete for a finite budget. It counts QUIC headers, control traffic, and retransmissions, not just useful media. It is an application-controlled bottleneck rather than a physical-link emulator or fair scheduler.

Both policies use a three-second subgroup forwarding limit. Termination ends the subscription with `TOO_FAR_BEHIND`. Skip resets pending media streams for that subscription and waits for a fresh group. The client explicitly permits stale-media dropping and stays on its original

connection and subscription. This timer begins when the relay's downstream forwarder observes a subgroup, so it is not a complete implementation of per-Object delivery deadlines or a strict memory limit.

We record complete Object delivery, stream resets, subscription terminations, QUIC counters, relay memory and CPU, commands, and source revisions. Terminated clients remain in throughput denominators through the planned end of the run.

Implementation checks and reproducibility

Before collecting results, we tested the relay and client changes with real QUIC connections. These checks cover expiry while a stream is opening or waiting for acknowledgment, intentional resets in the middle of an Object, delivery of a fresh group afterward, and rejection of malformed or truncated data. The upstream Rust tests, client tests, formatting and static checks passed, along with 18 Python tests for the experiment and analysis tools. These are correctness checks; the 115 accepted local runs provide the performance evidence.

We saved the source revision, patch and binary hashes, commands, raw event logs, and run-level summaries with each accepted dataset. Analysis retains clients that receive nothing or terminate, and reports recovery counts alongside recovery times. The repository also includes scripts to regenerate the figures. This gives us a repeatable local baseline to carry onto PACE.

Prepared first experiment on PACE ICE

We prepared a four-node launcher: one relay node, one publisher node, and two client-generator nodes. Each generator creates half of the independent subscriber connections. The script requests eight CPUs and 32 GiB per node for 30 minutes, without GPUs. These are allocation requests, not confirmed physical server specifications or a guarantee of exclusive nodes. We will record the actual CPU, memory, operating system, network interfaces, and routes from the allocated machines. [4, 5]

The launch script creates a certificate for the relay hostname, waits for relay and publisher readiness, starts the subscribers, collects process and protocol logs, and stops the other processes if a worker fails. A local integration test of all four worker roles passed using real QUIC traffic. This validates the coordination logic on one machine; it does not validate ICE allocation, software availability, or inter-node networking.

Before submission, we need to confirm our course's ICE allocation and shared working directory, build the pinned relay and client as Linux binaries in a compute allocation, and check node-to-node connectivity and clock offsets. The existing macOS binaries cannot be reused. We will run load tests on compute nodes, not the login node. The PACE guide records these steps and links to the ICE documentation; account-specific settings still need confirmation through GT sign-in. [4]

The first submitted run will be a 20-second baseline check with 16 subscribers, no pauses, and the stock relay. This separates cluster setup problems from policy behavior. After the Linux build, the prepared submission command is:

```
mkdir -p logs results
MOQT_CLIENTS=16 MOQT_DURATION=20 \
MOQT_STALL_CLIENTS=0 sbatch scripts/ice.slurm
```

We will accept the run only if all four workers exit successfully, all 16 clients have ready and complete records, payload verification succeeds, and the publisher sustains the intended source rate. A returned job number alone does not mean the experiment succeeded. The logs must also show generator CPU headroom and the actual network path. We will report cross-node one-way latency only after checking clock-offset uncertainty. [5]

After that check, the first repeated baseline comparison will use 16 clients with zero or eight paused for four seconds, five seconds after starting, followed by ten seconds of recovery. Both conditions will last 19 seconds and run five times in shuffled order. Eight clients run on each generator; the current placement puts the eight stalled clients on the first generator. We will keep placement fixed across comparisons and check its effect before scaling to larger client counts.

Readiness scope: the current ICE launcher selects the stock relay and does not enable the local 80 Mbit/s cap or the terminate/skip policies. Reproducing those comparisons on ICE still requires adding selection of the patched Linux relay/client, their policy arguments and build hashes, and adapting analysis to the two generator logs. DRR and detailed per-region memory measurements are also unfinished. This report stops at the prepared, locally validated baseline handoff; no PACE experiment has been submitted or completed.

Memory ownership and limits

To address the memory-isolation question from Report I, we traced the tested relay's storage and forwarding path. Figure 1 separates shared media from per-connection state. Our source is generated live, so there is no original file stored in the relay. Publisher memory and kernel socket buffers are outside the relay RSS we measure. [6]

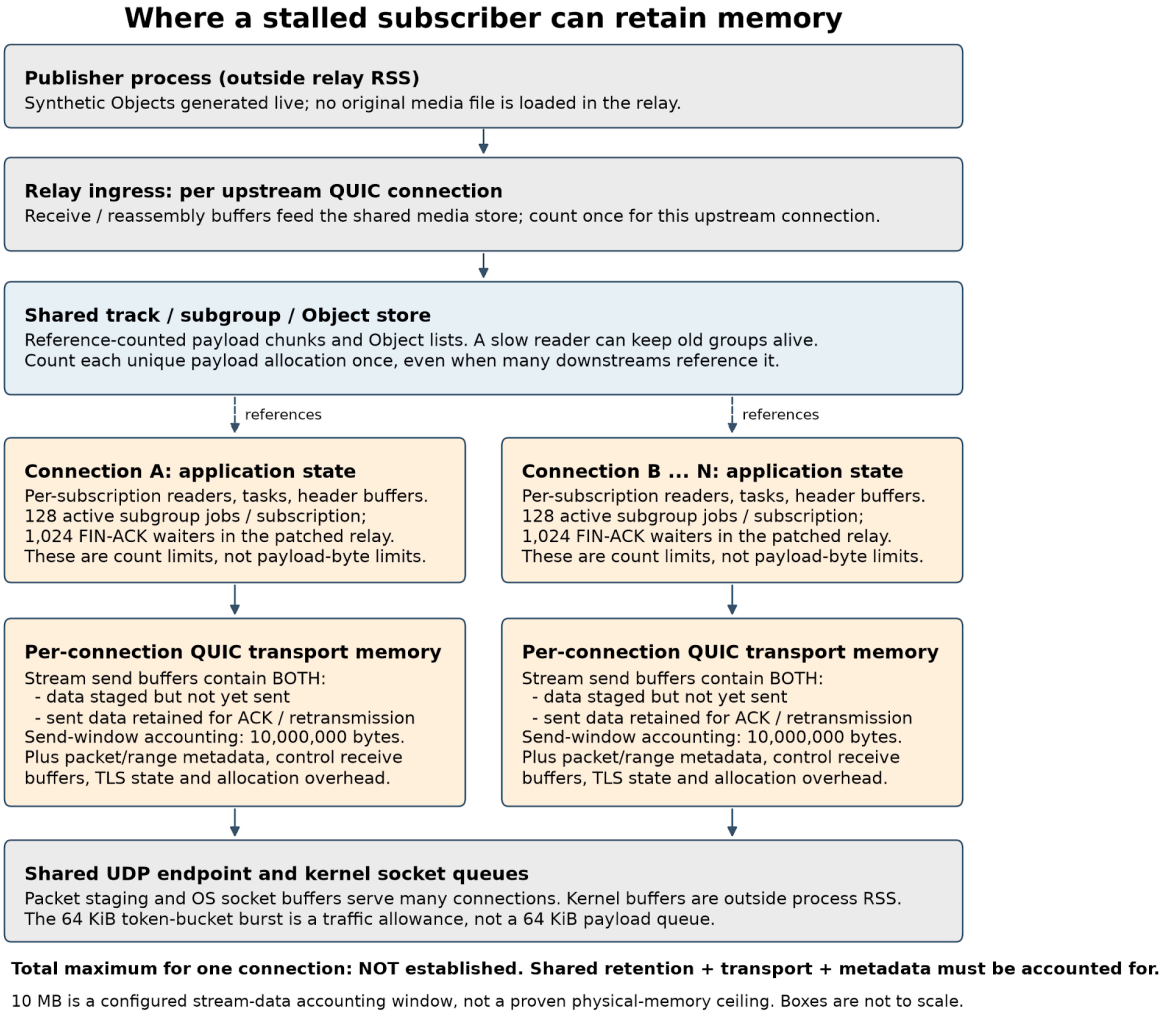


Figure 1. Memory ownership in the tested subgroup-stream path. Blue is shared media; orange is downstream connection state. The ingress region belongs to the upstream connection. Limits shown are configuration or job-count limits, not measured physical-memory peaks. A connection-wide maximum has not yet been established.

A stalled subscription holds references to shared subgroup data, so it can keep old Objects alive even after healthy clients finish reading them. We must count each unique payload allocation once. The patched forwarder permits 128 active subgroup jobs and 1,024 FIN-acknowledgment waiters per subscription, but these count limits do not bound group size or total bytes. The three-second timeout is also not a strict memory budget. [6]

The relay inherits Quinn's 10,000,000-byte send window (about 9.54 MiB) per QUIC connection. This limits outstanding stream-data accounting, not all connection memory. Unsent and sent-but-unacknowledged data occupy the same stream-buffer structure; they should not be counted as two full copies. Metadata, allocation overhead, and data retained behind acknowledgment gaps also matter. The subscriber's 64 KiB receive window does not cap the shared application data kept alive at the relay. [6, 7]

For scale, our source produces 499,980 payload bytes per one-second group. If all 128 active jobs retain one full group each, their referenced payload is about 64 MB (61 MiB), excluding other regions. This is a conditional calculation, not an observed peak or a total-connection bound. A 15-second segment contains about 7.5 MB; eight clients retaining that same segment do not create eight separate copies in the shared store. Independent tracks or disjoint retained groups can add memory. Similarly, 100 independently full 10 MB transport windows represent 1 GB of logical stream data, but our small-window clients with ongoing ACKs did not demonstrate that condition.

Our four-second runs measured median peak relay RSS of 26.98 MiB without skipping and 23.81 MiB with skipping. These are whole-process measurements. They do not establish a maximum per connection, memory exhaustion, or guaranteed memory isolation. The current evidence is strongest for bandwidth interference and recovery.

The next memory experiment will begin with one stalled subscriber plus healthy clients, varying pause length and group size before increasing concurrency. We will compare shared-track and distinct-track traffic and measure unique retained payload bytes, per-subscription references, QUIC accounting, heap retention, and RSS separately. A separate delayed/lost-ACK test will stress transport retention. We will choose larger client counts from the measured incremental memory use and available node memory, rather than assume that every connection consumes 1 MB or 1 GB.

5. Related Work

PicNIC: Predictable Virtualized NIC isolates virtual machines using CPU-fair queues, receiver-driven congestion control, and sender-side shaping and accounting. Its relevant lesson is that bandwidth fairness alone does not protect every shared resource. We apply that question to media sessions, where stale data loses value. Our current work adds accounting and fixed lifetime policies; it does not implement PicNIC's scheduling design. [1]

Media over QUIC: Initial Testing, Findings and Results provides an early experimental foundation for MoQ. The authors' companion testbed extends Warp with feedback and bandwidth measurements. Our testbed instead focuses on subscribers sharing a relay and the effect of stalled-client recovery. Encoded-video experiments remain future work. [2]

Efficient Fair Queuing Using Deficit Round-Robin provides an efficient way to share transmission service across queues. We plan to implement byte-based DRR as a fairness baseline and compare it with DRR plus stale-media limits. Fair byte allocation and useful live delivery are different questions: a client can receive its share while remaining far behind the source. DRR is not included in the current implementation or results. [3]

6. Evaluation Results and Next Steps

We completed 115 accepted local runs across three experiments. Delivery latency runs from publisher generation to a complete Object read. Timely goodput counts payload arriving within 150 ms, our experimental target. Throughput includes every expected client, including clients

receiving nothing. Results below are medians of five run-level measurements; they are not pooled across all Objects.

Our first 20 runs used 16 or 64 clients with no outgoing cap. Healthy clients stayed close to the source rate, and we found no consistent healthy-client degradation. Previously paused clients received about 10 Mbit/s during catch-up, but much of that traffic was late. This motivated adding a bandwidth bottleneck rather than claiming the original test already demonstrated unfairness.

The next 20 runs used 16 clients and the 80 Mbit/s cap, with zero or eight clients paused for 15 seconds. A three-second termination timeout reduced healthy-client recovery P99 from 190 to 11 ms and peak relay RSS from 36.92 to 23.70 MiB. However, it terminated all 40 paused subscriptions across the five mixed runs. This protected healthy clients by giving up on the stalled ones.

Our latest experiment tests whether clients can recover without disconnecting. We completed 75 runs: three policies (timeout disabled, terminate, and skip), four pause lengths (0.25, 1, 2, and 4 seconds), and five repetitions, plus five healthy-only controls per policy. Each mixed run has five seconds before the pause and ten seconds after it, with eight of 16 clients paused. Every condition uses the same relay and client builds and the same 80 Mbit/s cap.

To measure return to live, we look for the first complete one-second window after actual resume in which all received Objects meet the 150 ms target, at least 80% of the expected Objects arrive, and no delivery gap exceeds three frame intervals. Recovery time is the start of that window, confirmed using the following second. Clients without such a window before the ten-second observation limit are counted as unrecovered, not silently dropped.

Pause	Baseline recovered	Baseline recovery time	Skip recovered	Skip recovery time
0.25 s	40/40	0.088 s	40/40	0.080 s
1 s	40/40	1.733 s	40/40	1.693 s
2 s	40/40	4.104 s	40/40	3.979 s
4 s	30/40	8.443 s	40/40	0.024 s

Table 1. Recovery counts include all eight paused clients in each of five runs. Times are medians of per-run client medians, conditional on recovery; the four-second baseline excludes ten clients that did not recover within the observation window.

At four seconds, skip recovered all 40 paused clients on their original connections and subscriptions, with no terminations. The relay recorded 40 resynchronizations and clients recorded 40 intentional stream resets. Healthy-client P99 fell from 174 to 48 ms, while healthy timely goodput increased from 3.91 to 4.00 Mbit/s per client. Paused clients' timely goodput during the final five seconds was approximately 4.00 Mbit/s with skip, versus 1.30 Mbit/s with the baseline.

The 24 ms recovery median needs context: skip's five run medians ranged from 5 ms to 2.104 seconds, and the slowest individual client took 2.398 seconds. This is promising preliminary evidence, not a guarantee of near-instant recovery. Termination achieved a lower healthy P99 of 13 ms in this condition, but closed all 40 paused subscriptions and delivered no recovery traffic to them.

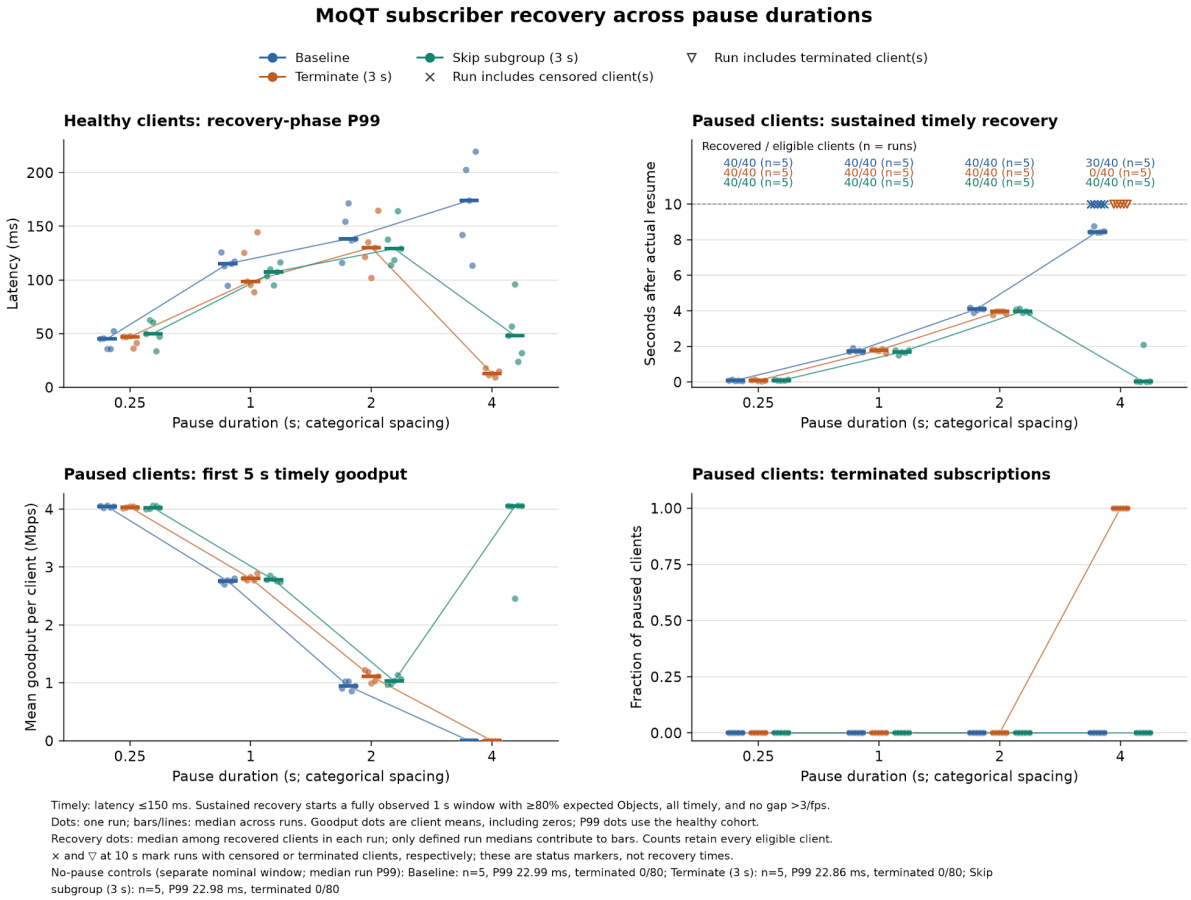


Figure 2. All short-pause repetitions. Recovery counts include every paused client; timing points include only clients that recovered. Markers at ten seconds identify runs with unrecovered or terminated clients, not measured recovery times.

No policy reset or terminated a subscription during the shorter pauses. All clients recovered, but two-second pauses still required about four seconds of catch-up. The skip policy helps the four-second pause because it crosses the three-second lifetime threshold and abandons old work. It does not automatically help shorter stalls. This is a useful limitation of a fixed threshold and motivates comparing more deliberate scheduling and backlog policies.

All 75 runs passed duration checks, covering 1,200 client observations. Connection and subscription identifiers remained consistent, and no healthy client was terminated. Source delivery during recovery stayed at 29.8-30.1 Objects/s. We retained raw measurements, hashes, and every repetition. Earlier attempts with large timing gaps remain separately marked as excluded.

These experiments use synthetic, independently usable groups on one machine. Real video may need keyframes before playback can resume; resetting a stream is not by itself proof of successful video recovery. We also have not tested higher incoming-stream concurrency, upstream subscription sharing, or separate-node network conditions. The timer is a fixed forwarding lifetime, not an adaptive buffer budget.

Our immediate next experimental step is the prepared PACE baseline check, followed by the repeated baseline comparison after the cluster checks pass. The next implementation step is byte-based DRR across subscriber sessions and support for the modified policies in the cluster launcher. We will compare DRR alone with DRR plus stale-media skipping under the same cap, measuring per-session throughput and Jain's fairness index alongside timely goodput, latency, and recovery. Multi-track subscribers, encoded media, and the memory-accounting experiments described above remain future work. The current results demonstrate contention and recovery without disconnecting; they do not yet demonstrate fair scheduling or a per-connection memory bound.

References

1. Kumar et al. **PicNIC: Predictable Virtualized NIC.** SIGCOMM 2019. Paper.

2. Gurel et al. **Media over QUIC: Initial Testing, Findings and Results.** MMSys 2023. DOI; authors' testbed.

3. Shreedhar and Varghese. **Efficient Fair Queuing Using Deficit Round-Robin.** IEEE/ACM Transactions on Networking, 1996. Paper.

4. Georgia Tech PACE. ICE job guide and storage guide, linked from the GT teaching tutorial.

5. SchedMD. sbatch and srun documentation.

6. Project memory audit: ownership, source anchors, limits, and scale calculations for the tested relay.

7. Quinn-proto 0.11.13: transport configuration and send-buffer implementation.

Project repository